

Complete Concept Breakdown, Implementation Mapping & Interview Answers

JWT DUAL-TOKEN ROTATION

[illegible]

ðŸŽ– Interview Answer: "We use Mongoose schemas to ensure schema validation at the database layer. By using `select: false` on passwords and tokens, we eliminate accidental credential leaks in API responses."

2. Mongoose Lifecycle Hooks (Pre & Post Middleware)

models/User.model.js, Post.model.js, Comment.model.js

What & Why: Executes automated business logic before or after document operations (save, delete) to maintain data consistency without bloating controllers.

- **Pre-save (User):** Automatically hashes user passwords with `bcryptjs` only when the password field is created or modified (`this.isModified('password')`).
- **Pre-save (Post):** Calculates estimated reading time dynamically based on word count (`words / 200 wpm`).
- **Post-save (Comment):** Automatically executes `Post.findByIdAndUpdate(doc.post, { $inc: { commentsCount: 1 } })`.
- **Post-deleteOne (Comment):** Automatically decrements `commentsCount` when a comment is removed.

ðŸŽŒ Interview Answer: "We use Mongoose pre/post middleware for separation of concerns. Instead of manually updating comment counters or hashing passwords in multiple controllers, the database models manage their own lifecycle logic."

3. Mongoose Virtuals & Schema Transformations

models/User.model.js, Post.model.js, Comment.model.js

What & Why: Computed properties that are dynamically calculated from document properties and included in JSON responses without storing duplicate data in MongoDB.

```
userSchema.virtual('followerCount').get(function () { return this.followers ? this.followers.length : 0; });
userSchema.virtual('followingCount').get(function () { return this.following ? this.following.length : 0; });
// Configured with: { toJSON: { virtuals: true }, toObject: { virtuals: true } }
```

ðŸŽŒ Interview Answer: "Virtuals prevent database redundancy. Rather than storing and synchronizing a separate `followerCount` integer field, we compute it on the fly from the `followers` array."

4. Schema Methods (Document Instance Methods)

models/User.model.js

What & Why: Custom helper methods bound directly to Mongoose document instances.

- `userSchema.methods.comparePassword(plainText)`: Encapsulates `bcrypt` verification against `this.password`.
- `userSchema.methods.toPublicProfile()`: Strips sensitive fields (tokens, internal IDs) and returns clean profile JSON.

5. Relationship Referencing & Mongoose .populate()

controllers/post.controller.js, comment.controller.js

What & Why: MongoDB is non-relational, but we store `mongoose.Schema.Types.ObjectId` references with `ref: 'User'` or `ref: 'Post'`. `.populate()` acts like a SQL JOIN, replacing the ID with the referenced document.

```
const posts = await Post.find(query)
  .sort({ createdAt: -1 })
  .skip(skip).limit(limit)
  .populate('author', 'username name avatar') // Only selects needed user fields
  .lean(); // Converts to plain JS object for performance
```

ðŸŽŒ Interview Answer: "We use selective field population (`'username name avatar'`) combined with `.lean()` to avoid fetching unnecessary data and eliminate Mongoose hydration overhead for read-heavy routes."

6. Database Indexing, Compound Indexes & TTL Auto-Expiry

models/Post.model.js, User.model.js,
Notification.model.js

What & Why: High-speed query execution and automated document cleanup.

- **Full-Text Search Index:** `postSchema.index({ title: 'text', content: 'text', tags: 'text' })` and `userSchema.index({ name: 'text', username: 'text', skills: 'text' })` for weighted search queries with `score: { $meta: 'textScore' }`.
- **Compound Index:** `notificationSchema.index({ recipient: 1, isRead: 1, createdAt: -1 })` speeds up unread notification queries.
- **TTL (Time-To-Live) Index:** `notificationSchema.index({ createdAt: 1 }, { expireAfterSeconds: 30 * 24 * 60 * 60 })`. MongoDB automatically removes notifications older than 30 days without any cron jobs.

🔗 Interview Answer: "We implemented a MongoDB TTL index on the notifications collection so old alerts are automatically purged by the database background engine after 30 days."

7. Atomic Update Operators (\$addToSet, \$pull, \$inc, \$in)

controllers/post.controller.js, user.controller.js

What & Why: Executes atomic operations in MongoDB without race conditions or read-modify-write cycles.

- `$addToSet`: Adds user ID to `likes`, `bookmarks`, or `following` array only if not already present.
- `$pull`: Removes user ID from array upon unliking or unfollowing.
- `$inc`: Atomically increments view counts (`{ $inc: { views: 1 } }`) and comment counts.
- `$in`: Fetches personalized home feed by matching post authors against followed users (`{ author: { $in: followingIds } }`).

3. Express.js & Middleware Architecture (E in MERN)

1. Layered RESTful Routing & Express Router

server.js, routes/*.routes.js

What & Why: Modularizes endpoints into domain-specific routers: `/api/v1/auth`, `/api/v1/users`, `/api/v1/posts`, `/api/v1/comments`, and `/api/v1/notifications`.

2. Custom Authentication Middleware (verifyJWT) & Ownership Verification

middleware/auth.middleware.js

What & Why: Protects private endpoints by decoding Bearer JWT tokens from the `Authorization` header and attaching the user object to `req.user`.

```
export const verifyJWT = asyncHandler(async (req, _res, next) => {
  const authHeader = req.headers.authorization;
  if (!authHeader?.startsWith('Bearer ')) throw ApiError.unauthorized('Token missing');
  const token = authHeader.split(' ')[1];
  const decoded = jwt.verify(token, process.env.JWT_ACCESS_SECRET);
  const user = await User.findById(decoded.id).select('-password -refreshToken');
  if (!user) throw ApiError.unauthorized('User not found');
  req.user = user;
  next();
});

// Middleware Factory: High-order function to verify resource ownership
export const checkOwnership = (getOwnerId) => asyncHandler(async (req, _res, next) => {
  if (getOwnerId(req) !== req.user._id.toString()) throw ApiError.forbidden('Unauthorized action');
  next();
});
```

3. Global Centralized Error Handling Middleware

middleware/error.middleware.js, utils/ApiError.js

What & Why: Captures all synchronous and asynchronous errors in a single 4-argument Express error handler (`err`, `req`, `res`, `next`). Translates Mongoose `CastError` (400), `ValidationError` (422), Mongo duplicate key error code 11000 (409), and JWT errors (401) into clean, standardized JSON errors.

🔗 Interview Answer: "We use a custom `ApiError` class and a global error middleware registered last in Express. This eliminates repetitive error formatting across controllers and hides internal stack traces in production."

4. Controller Async Wrapper (asyncHandler)

utils/asyncHandler.js

What & Why: A higher-order function that wraps asynchronous Express handlers in `Promise.resolve().catch(next)`, eliminating repetitive `try/catch` blocks in every controller.

5. Declarative Input Validation with express-validator

routes/auth.routes.js, middleware/validate.middleware.js

What & Why: Validates and sanitizes incoming HTTP request body/params before the controller runs. If validation fails, `validate.middleware.js` intercepts the result and responds immediately with status 400 and structured error arrays.

ðŹŹŹ 4. React 19 & Frontend State Architecture (R in MERN)

1. Global Auth State with Context API & useReducer

frontend/src/context/AuthContext.jsx

What & Why: Manages authentication state (`user` , `isAuthenticated` , `isLoading`) across the component tree without prop drilling. Employs a pure Redux-style reducer with action types: `LOGIN` , `LOGOUT` , `UPDATE_USER` , and `SET_LOADING` .

Silent Session Restoration: On initial app load (`useEffect`), `AuthContext` automatically calls `/auth/refresh` to restore user sessions from the `httpOnly` refresh cookie.

ðŹŹŹ Interview Answer: "We pair Context API with `useReducer` instead of multiple `useState` hooks to ensure predictable, atomic auth state transitions across the application."

2. Server State & Cache Management with TanStack React Query v5

frontend/src/hooks/usePosts.js, useUsers.js, useComments.js

What & Why: Separates server data caching from local UI state. Handles caching, background refetching, query invalidation, deduplication, and loading states automatically.

• `useQuery` : Fetches and caches data with structured query key factories (e.g. `postKeys.list(filters)`).

• `useMutation` : Handles POST/PUT/DELETE actions and automatically invalidates stale queries via `queryClient.invalidateQueries()` on success to instantly refresh UI lists.

• `useInfiniteQuery` : Drives smooth infinite-scroll pagination on the home feed.

ðŹŹŹ Interview Answer: "React Query drastically reduced our frontend boilerplate. We don't write manual loading flags or `useEffect` fetch loops for API data â€” React Query manages the cache lifecycle."

3. Protected Routes & Declarative Routing (React Router v6)

frontend/src/App.jsx, routes/ProtectedRoute.jsx

What & Why: `ProtectedRoute` guards private pages (Create Post, Notifications, Bookmarks). If unauthenticated, it redirects to `/login` while preserving the intended destination URL via router state.

4. High-Performance Form Handling with react-hook-form & Zod

frontend/src/pages/Register.jsx, Login.jsx, Profile.jsx

What & Why: Employs uncontrolled inputs via `register()` to eliminate re-renders on every keystroke, validated by strict, type-safe `Zod` schemas connected via `@hookform/resolvers/zod` .

ðŹŹŹ 5. Node.js & Asynchronous Programming (N in MERN)

1. Native ECMAScript Modules (ESM)

backend/package.json ("type": "module"), server.js

What & Why: Standard modern module syntax (`import` / `export`) across both backend and frontend, enabling top-level imports and clean tree-shaking.

2. Parallel Execution with Promise.all()

controllers/post.controller.js, user.controller.js

What & Why: Runs independent asynchronous database queries concurrently on Node's event loop to halve response latency.

```
// Executes post query and total count in parallel
const [posts, total] = await Promise.all([
  Post.find(query).sort(sortObj).skip(skip).limit(limit).lean(),
  Post.countDocuments(query)
]);
```

3. Cryptographic Token Generation (Node.js Crypto Module)

controllers/auth.controller.js

What & Why: Generates secure 20-byte random hex tokens for password resets (crypto.randomBytes(20).toString('hex')) and hashes them with SHA-256 before saving to the database.

ðŸ”Œ 6. Security, Authentication & Session Architecture

1. Dual-Token Architecture & Refresh Token Rotation

utils/generateToken.js, controllers/auth.controller.js

What & Why: Protects against both XSS and CSRF token theft.

- **Access Token (15 min):** Short-lived JWT containing user payload, stored strictly in React memory (module variable in `axios.js`). Sent via `Authorization: Bearer` header.
- **Refresh Token (7 days):** Long-lived JWT stored in an `httpOnly`, `sameSite: 'strict'`, `secure` cookie and hashed/saved in MongoDB.
- **Token Rotation:** Every call to `/auth/refresh` replaces the stored refresh token with a new one. If an old token is presented again, the server identifies token reuse and revokes access.

2. Axios Interceptor Queue for Automatic Token Refresh

frontend/src/api/axios.js

What & Why: When an API request fails with a `401 Unauthorized` (access token expired), the Axios response interceptor intercepts the error, calls `/auth/refresh`, updates the access token, and transparently retries the original request.

Request Queuing: If multiple API calls fail simultaneously with 401s, they are pushed into a `failedQueue` and resolved once the single in-flight refresh request completes.

ðŸ”Œ Interview Answer: "Our Axios interceptor implements a request queue pattern. It silently refreshes the JWT access token without logging the user out or triggering multiple redundant refresh requests."

3. HTTP Security Headers (Helmet) & CORS Configuration

server.js

What & Why: `helmet()` sets 14 protective HTTP headers (Content-Security-Policy, X-Frame-Options against clickjacking, X-Content-Type-Options). `cors()` restricts API access exclusively to `process.env.CLIENT_ORIGIN` with `credentials: true` for secure cookie transport.

ðŸ”Œ 7. Real-Time Communication (Socket.io)

1. WebSocket Handshake JWT Authentication

socket/socket.js, frontend/src/context/SocketContext.jsx

What & Why: Socket.io connection attempts pass through an authentication middleware:

```
io.use(async (socket, next) => {
  const token = socket.handshake.auth?.token;
  if (!token) return next(new Error('Auth token missing'));
  const decoded = jwt.verify(token, process.env.JWT_ACCESS_SECRET);
  socket.user = await User.findById(decoded.id).select('_id username name avatar').lean();
  next();
});
```

2. Private User Rooms & Targeted Notifications

socket/socket.js, controllers/post.controller.js

What & Why: Upon connection, each user joins a private Socket.io room named after their unique user ID (`socket.join(userId)`). When a user receives a like, comment, or follow, the controller accesses the socket instance via `req.app.get('io')` and emits the notification directly to that user's private room:
`io.to(recipientId).emit('notification:new', payload)`.

3. Live Online Presence Tracking

socket/socket.js, frontend/src/context/SocketContext.jsx

What & Why: Broadcasts `user:online` and `user:offline` events across all connected clients on connection/disconnection and updates `isOnline` & `lastSeen` in MongoDB.

ðŸ“ˆ 8. File Uploads & Cloud Media (Multer + Cloudinary)

1. In-Memory File Buffering with Multer

middleware/upload.middleware.js

What & Why: Uses `multer.memoryStorage()` to retain uploaded files in RAM as a Buffer with strict MIME-type validation (JPEG, PNG, WebP) and a 5MB size limit, avoiding temporary disk writes.

2. Direct Cloudinary Buffer Streaming & Transformations

controllers/user.controller.js, post.controller.js

What & Why: Uses `streamifier` to pipe the memory buffer directly into `cloudinary.uploader.upload_stream()`, applying automated on-the-fly transformations (e.g. `crop: 'fill', gravity: 'face', width: 400, height: 400` for profile avatars).

🔗 9. Quick Interview Reference Cheat Sheet

Concept	Why It Was Used in DevConnect	How to Explain in an Interview
JWT Dual-Token Pattern	Secure auth without storing long-lived credentials in JS-accessible storage.	"We store access tokens in memory for speed and refresh tokens in <code>httpOnly</code> cookies with rotation to defend against XSS and CSRF."
Axios Interceptor Queue	Handles 401 token expiry seamlessly in the background.	"If an access token expires mid-session, Axios queues outgoing requests, fetches a new token via refresh cookie, and replays requests without user interruption."
TanStack Query v5	Caches server responses and handles optimistic UI and cache invalidation.	"Eliminates boilerplate <code>useEffect</code> data fetching. We use structured query keys and invalidate stale queries on mutations."
Mongoose Hooks & Virtuals	Separation of business logic (password hashing, comment counting, read time).	"Pre-save hooks hash modified passwords, post-save hooks keep relational counters in sync, and virtuals prevent storing redundant derived values."
MongoDB TTL Indexes	Automated notification purge after 30 days.	"MongoDB's background TTL thread auto-purges documents past the expiry timestamp, eliminating the need for scheduled cron jobs."
Socket.io Private Rooms	Instant, targeted alerts for likes, comments, and follows.	"Each authenticated socket joins a room keyed by user ID. Controllers emit real-time events directly to the recipient's room."
Multer + Cloudinary Stream	Profile avatar and post banner image uploads.	"We use Multer memory storage and stream the buffer directly to Cloudinary with facial-detection cropping, avoiding local disk writes."
Global Error Handling	Unified error responses and status code mapping.	"All controllers use <code>asyncHandler</code> and throw custom <code>ApiError</code> instances. The central error middleware standardizes responses."
Docker Compose Deployment	Orchestrates MongoDB 7, Express API, and Nginx-served React frontend.	"Multi-stage Docker builds provide lightweight production images, and Docker Compose configures service health checks and network bridging."